IBM Developer
SKILLS NETWORK

Winning Space Race with Data Science

Mamady Magassouba
2025-11-16

Outline

- Executive Summary
- Introduction
- Methodology
- Results
- Conclusion
- Appendix

Executive Summary

This project analyzes SpaceX Falcon 9 and Falcon Heavy launch records to uncover insights into launch outcomes, site selection, and payload success. Using data collection, web scraping, exploratory data analysis, SQL, geospatial visualization, and machine learning, we identify key factors influencing launch success and provide recommendations for future launches.

Executive Summary

Summary of methodologies

Data was collected from the SpaceX API and Wikipedia using Python scripts and web scraping.

- Data wrangling and cleaning were performed to handle missing values and unify formats.
- EDA was conducted using Pandas, SQL, and visualization libraries.
- Interactive analytics were developed with Folium (for maps) and Plotly Dash (for dashboards).
- Predictive models (Logistic Regression, SVM, Decision Trees, KNN) were built, tuned, and evaluated.

Summary of all results

Identified key features (payload mass, launch site, booster version) affecting landing success.

- Created interactive dashboards and maps for visual analytics.
- Achieved over 80% accuracy in predicting landing outcomes using tuned classification models.

Introduction

- **Project background and context**

  SpaceX's reusable rocket program has revolutionized spaceflight economics. Predicting the success of Falcon 9 first stage landings is crucial for cost savings and mission planning. This project aims to analyze historical launch data to uncover patterns and build predictive models for landing outcomes.

- Problems you want to find answers

  What factors most influence the success or failure of Falcon 9 landings?

- Can we predict the outcome of a launch based on available features?
- How can interactive analytics and geospatial visualization help stakeholders understand launch patterns?

Section 1

Methodology

Methodology

Executive Summary

- Data collection methodology:
  - API Data: Used Python's requests library to collect launch data from the SpaceX API, including rocket, payload, launchpad, and core details.
  - Web Scraping: Employed BeautifulSoup to extract historical launch records from Wikipedia tables.
  - Data Export: Saved processed data to CSV files for reproducibility and further analysis.
- Perform data wrangling
  - Merged API and web-scraped data.
  - Cleaned missing values, standardized column names, and filtered for Falcon 9 launches.
  - Engineered new features (e.g., landing outcome labels, class variable for success/failure).
- Perform exploratory data analysis (EDA) using visualization and SQL
- Perform interactive visual analytics using Folium and Plotly Dash
- Perform predictive analysis using classification models

Data Collection

Data Sources and Process

- SpaceX API: Automated data extraction for launch details, rocket specs, and outcomes.
- Wikipedia: Scraped tabular data for historical launches and outcomes.

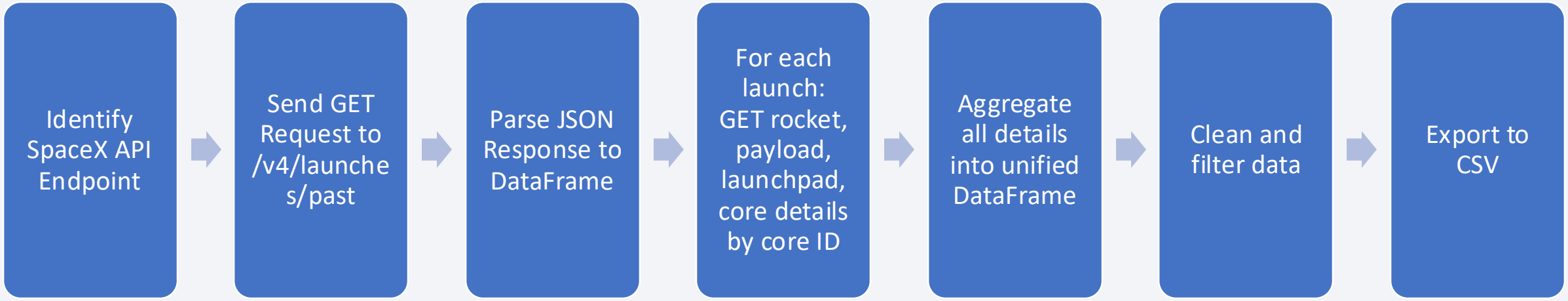
```
[SpaceX API] -----\
                   >--- [Data Wrangling & Cleaning] ---> [Unified Dataset] ---> [EDA/Modeling]
[Wikipedia]  -----/
```

Data Collection – SpaceX API

- **Identify API Endpoint:**
  Use the SpaceX API endpoint https://api.spacexdata.com/v4/launches/past to retrieve historical launch data.
- **Send HTTP GET Request:**
  Use Python's requests.get() to fetch data from the API.
- **Parse JSON Response:**
  Convert the JSON response into a pandas DataFrame for analysis.
- **Extract Related Data via IDs:**
  For each launch, use IDs (for rocket, payload, launchpad, cores) to make additional API calls and retrieve detailed information.
- **Aggregate and Clean Data:**
  Combine all extracted information into a unified DataFrame, handle missing values, and filter for Falcon 9 launches.
- **Export Data:**
  Save the cleaned DataFrame to a CSV file for further analysis.

Data Collection – SpaceX API

flowchart of SpaceX API calls



- GitHub URL of the completed SpaceX API calls notebook:

  https://github.com/MamadyMagass/Applied-Data-Science-Capstone-IBM/blob/main/week_1/jupyter-labs-spacex-data-collection-api.ipynb

Data Collection - Scraping

- Identify Data Source: Selected the Wikipedia page for Falcon 9 and Falcon Heavy launches.
- Send HTTP Request: Used requests.get() with a User-Agent header to fetch the HTML content.
- Parse HTML: Utilized BeautifulSoup to parse the HTML and locate the target table.
- Extract Table Headers: Iterated through <th> elements to get column names.
- Extract Table Rows: Parsed <tr> and <td> elements to collect launch data.
- Clean Data: Handled missing values, removed unwanted characters, and standardized formats.
- Store Data: Compiled the extracted data into a pandas DataFrame.
- Export Data: Saved the cleaned DataFrame to a CSV file for further analysis.

Data Collection - Scraping

flowchart

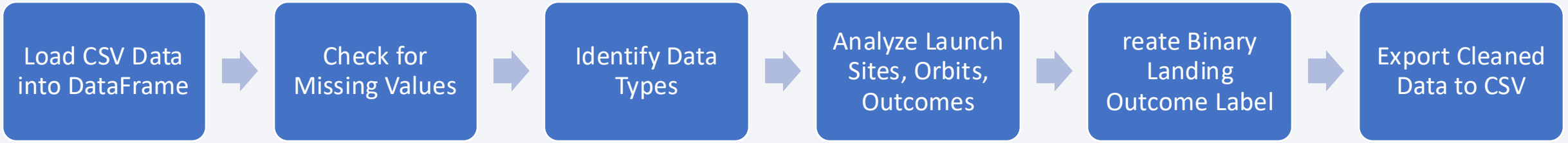
- GitHub URL of the completed web scraping notebook: https://github.com/MamadyMagass/Applied-Data-Science-Capstone-IBM/blob/main/week_1/jupyter-labs-webscraping.ipynb

Data Wrangling

- Data Loading: Imported the SpaceX Falcon 9 dataset using pandas.read_csv().
- Missing Value Analysis: Calculated the percentage of missing values in each column using df.isnull().sum()/len(df)*100.
- Data Type Identification: Identified numerical and categorical columns with df.dtypes.
- Exploratory Data Analysis: Used value_counts() to analyze launch sites, orbits, and mission outcomes.
- Label Creation: Created a binary classification label (Class) for landing success based on the Outcome column.
- Data Export: Saved the cleaned and labeled DataFrame to a CSV file for further analysis.

Data Wrangling

flowchart for Data wrangling



- GitHub URL of the completed Datawrangling notebook:

https://github.com/MamadyMagass/Applied-Data-Science-Capstone-IBM/blob/main/week_1/labs-jupyter-spacex-Data wrangling.ipynb

EDA with Data Visualization

Charts plotted :

- Flight Number vs. Payload Mass (Categorical Plot)

To observe how the number of launches and payload mass relate to landing success. This helps identify trends in experience and payload handling.

- Flight Number vs. Launch Site (Categorical Plot)

To analyze if launch experience at different sites affects success rates.

- Payload Mass vs. Launch Site (Categorical Plot)

To see if certain sites handle heavier payloads and how that impacts success.

- Success Rate by Orbit Type (Bar Chart)

To compare which orbit types have higher or lower landing success rates.

- Flight Number vs. Orbit Type (Scatter Plot)

To explore if experience (flight number) impacts success for different orbits.

- Payload Mass vs. Orbit Type (Scatter Plot)

To examine the relationship between payload mass and success across orbit types.

- Yearly Success Rate Trend (Line Plot)

To visualize how the overall success rate has changed over time.

EDA with Data Visualization

GitHub URL of the completed EDA with data visualization notebook :

- https://github.com/MamadyMagass/Applied-Data-Science-Capstone-IBM/blob/main/week_2/jupyter-labs-eda-dataviz-v2.ipynb

EDA with SQL

SQL queries performed :

- Selected unique launch site names from the missions.
- Displayed 5 records where launch sites begin with 'CCA'.
- Calculated the total payload mass for boosters launched by NASA (CRS).
- Computed the average payload mass for booster version F9 v1.1.
- Found the date of the first successful ground pad landing.
- Listed booster versions with successful drone ship landings and payload mass between 4000 and 6000 kg.
- Counted the total number of successful and failed mission outcomes.

EDA with SQL

- Identified booster versions that carried the maximum payload mass using a subquery.
- Displayed records of failed drone ship landings in 2015, including month, booster version, and launch site.
- Ranked landing outcomes by count for missions between 2010-06-04 and 2017-03-20.

GitHub URL of the completed EDA with SQL:

https://github.com/MamadyMagass/Applied-Data-Science-Capstone-IBM/blob/main/week_2/jupyter-labs-eda-sql-coursera_sqllite.ipynb

Build an Interactive Map with Folium

Map objects :

- **Circles:** Added folium.Circle objects at each launch site to visually highlight their locations on the map.
- **Markers:** Added folium.Marker and DivIcon objects to label each launch site and display their names.
- **Colored Markers:** Placed green markers for successful launches and red markers for failed launches to visualize launch outcomes at each site.
- **Marker Clusters:** Used MarkerCluster to group overlapping markers for better map readability.
- **Mouse Position:** Added MousePosition plugin to easily get coordinates of points of interest on the map.
- **Distance Markers:** Placed markers at nearby features (coastline, city, railway, highway) showing the distance from the launch site.
- **Polylines:** Drew folium.PolyLine lines between launch sites and nearby features to visualize and measure distances.

Build an Interactive Map with Folium

- **Reason:**
  These objects were added to visually analyze the geographic distribution of launch sites, understand their proximity to important features (like coastlines, cities, railways, highways), and explore how location might relate to launch outcomes. This interactive visualization helps reveal spatial patterns and relationships that are not obvious from raw data.

GitHub URL of Interactive Map with Folium :

https://github.com/MamadyMagass/Applied-Data-Science-Capstone-IBM/blob/main/week_3/lab-jupyter-launch-site-location-v2.ipynb

Build a Dashboard with Plotly Dash

Plots/graphs added to dashboard :

- The **dropdown menu** allows users to select a specific launch site or view all sites, making the dashboard flexible for different analysis needs.
- The **pie chart** provides a quick visual summary of launch success rates, either by site or overall, helping users compare performance at a glance.
- The **payload range slider** lets users filter launches by payload mass, enabling focused analysis on how payload size affects outcomes.
- The **scatter plot** shows the relationship between payload mass and launch success, colored by booster version, to help users identify trends and correlations.
- **Interactivity** ensures that all plots update dynamically based on user selections, making the dashboard more engaging and allowing users to explore the data from multiple perspectives.

Build a Dashboard with Plotly Dash

GitHub URL for Dashboard with Plotly Dash :

https://github.com/MamadyMagass/Applied-Data-Science-Capstone-IBM/blob/main/week_3/spacex-dash-app.py

Predictive Analysis (Classification)

Summary of Model Development Process:

- **Data Preparation:**
  - Loaded datasets and selected relevant features.
  - Standardized feature data using StandardScaler.
  - Split data into training and test sets (80/20 split).
- **Model Building:**
  - Built four classification models: Logistic Regression, Support Vector Machine (SVM), Decision Tree, and K-Nearest Neighbors (KNN).
  - Used GridSearchCV with cross-validation (cv=10) to tune hyperparameters for each model.
- **Model Evaluation:**
  - Evaluated each model using accuracy score on the test set.
  - Visualized results with confusion matrices to analyze true/false positives and negatives.

Predictive Analysis (Classification)

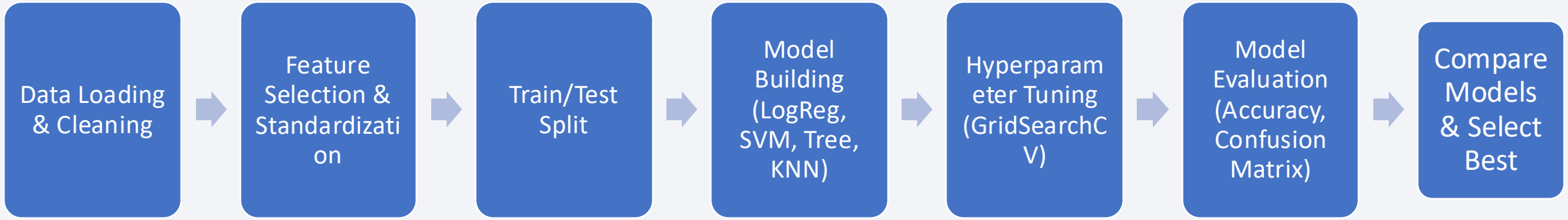
- **Model Improvement:**
  - Selected best hyperparameters from grid search for each model.
  - Compared test accuracies to identify the best performing model.
- **Model Selection:**
  - Chose the model with the highest test accuracy as the final predictive model.

GitHub URL for Predictive Analysis :

https://github.com/MamadyMagass/Applied-Data-Science-Capstone-IBM/blob/main/week_4/SpaceX_Machine Learning Prediction_Part_5.ipynb

Predictive Analysis (Classification)

flowchart for Predictive Analysis



Results

- Exploratory data analysis results
  - Launch Site Analysis: Most launches occur at coastal sites for safety and orbital efficiency.
  - Payload Trends: Higher payloads are associated with certain booster versions and orbits.
  - Outcome Distribution: Success rates vary by site and booster version; overall success rate is high.

Results

- Interactive analytics demo in screenshots
  - Folium Map: Visualizes launch site locations and infrastructure proximity.
  - Dash Dashboard: Allows users to filter by site and payload, updating pie and scatter charts interactively.

Results

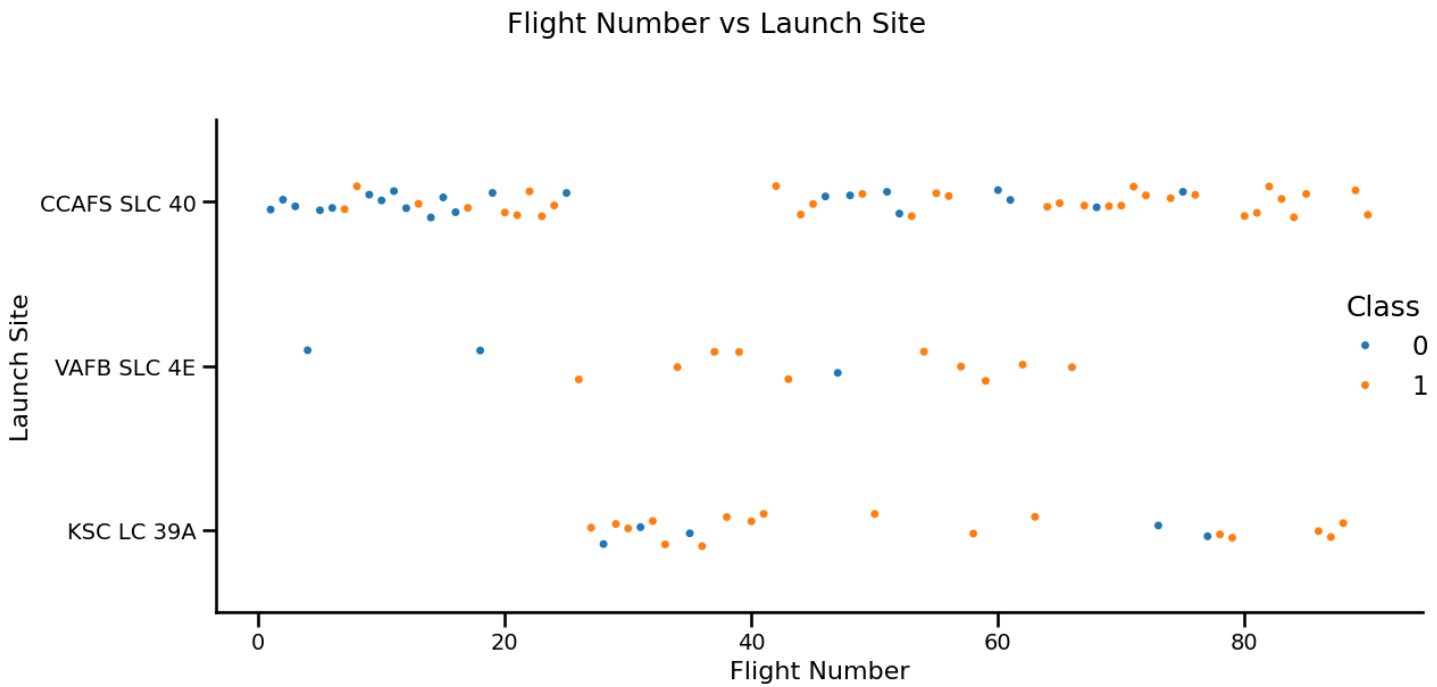
- Predictive analysis results
  - Best performing model on train data set is Decision Tree with ~ 0. 9 accuracy.
  - All models perform exactly same on test data set with 0.944 accuracy.
  - Feature Importance: ReusedCount, Payload mass, Legs_False, and Legs_True are key predictors.
  - Confusion Matrices: Showed good balance between precision and recall for success/failure classes.

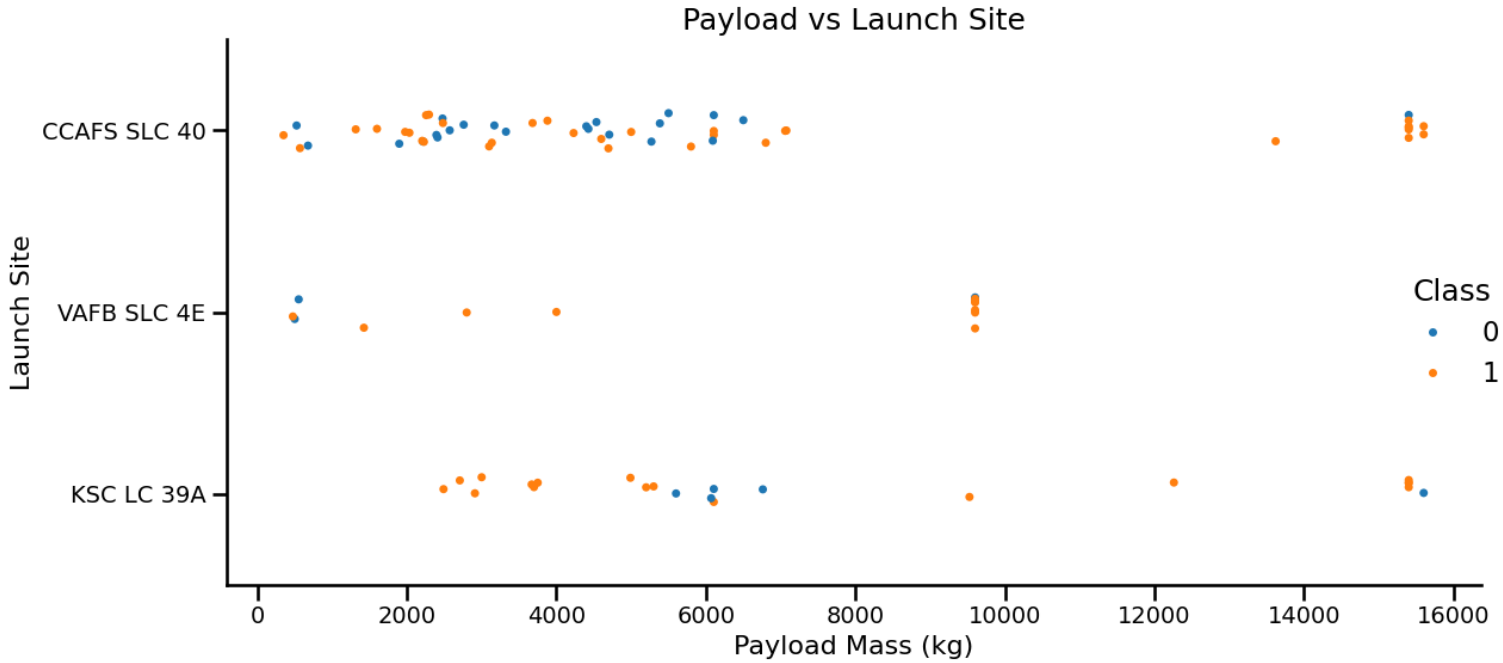
Section 2

Insights drawn from EDA

Flight Number vs. Launch Site

It seems that at every launch site, the higher the number of flights, the more successful the landing.



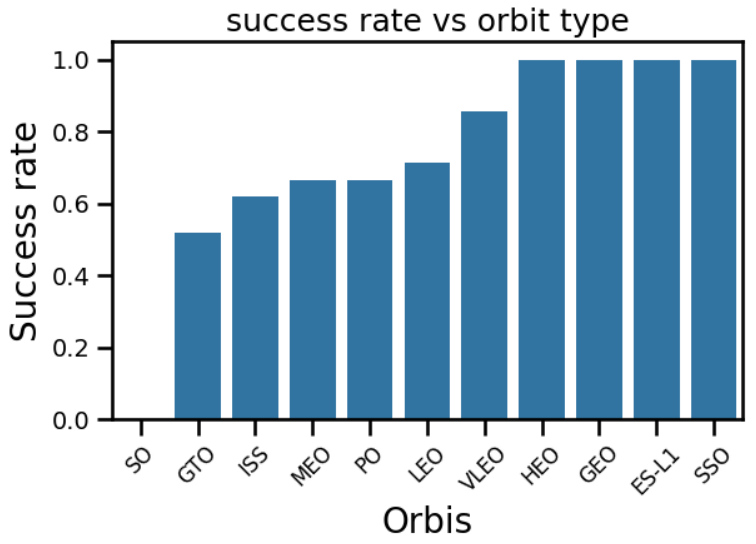


When we observe Payload Vs. Launch Site scatter point chart you will find for the VAFB-SLC launch site there are no rockets launched for heavy payload mass(greater than 10000).

Payload vs. Launch Site

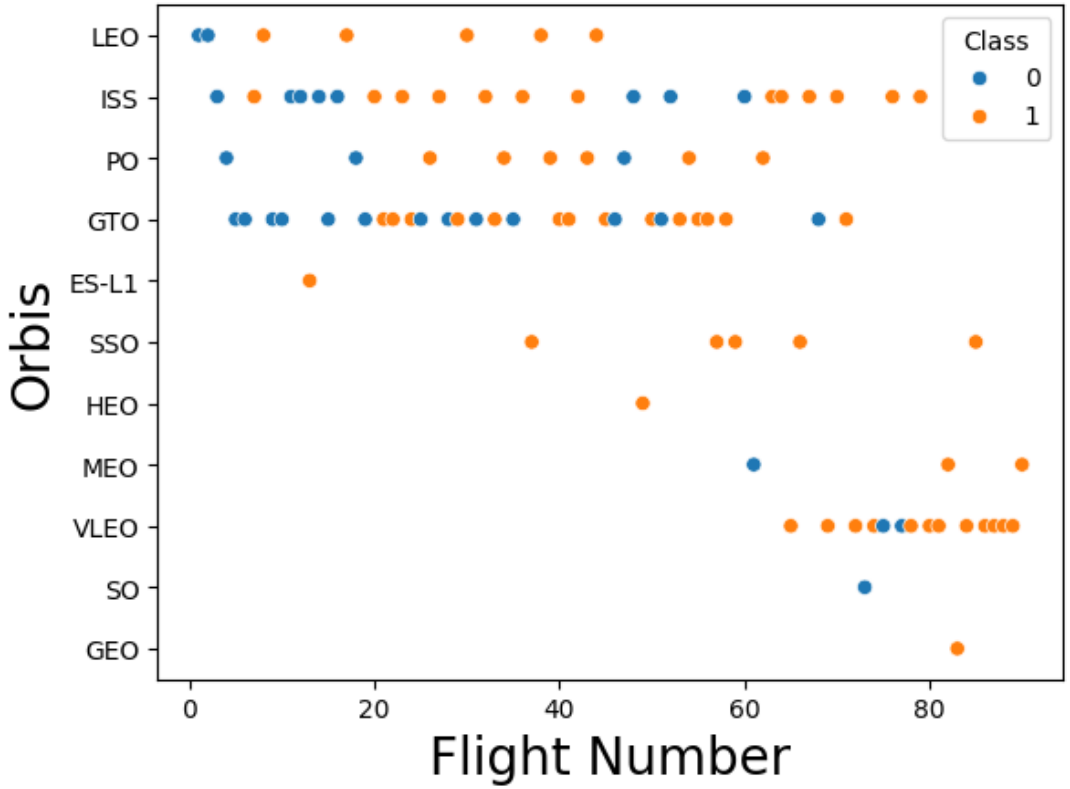
Success Rate vs. Orbit Type

The orbits which have sucessfull rate are : HEO, GEO, ES-L1, SSO.



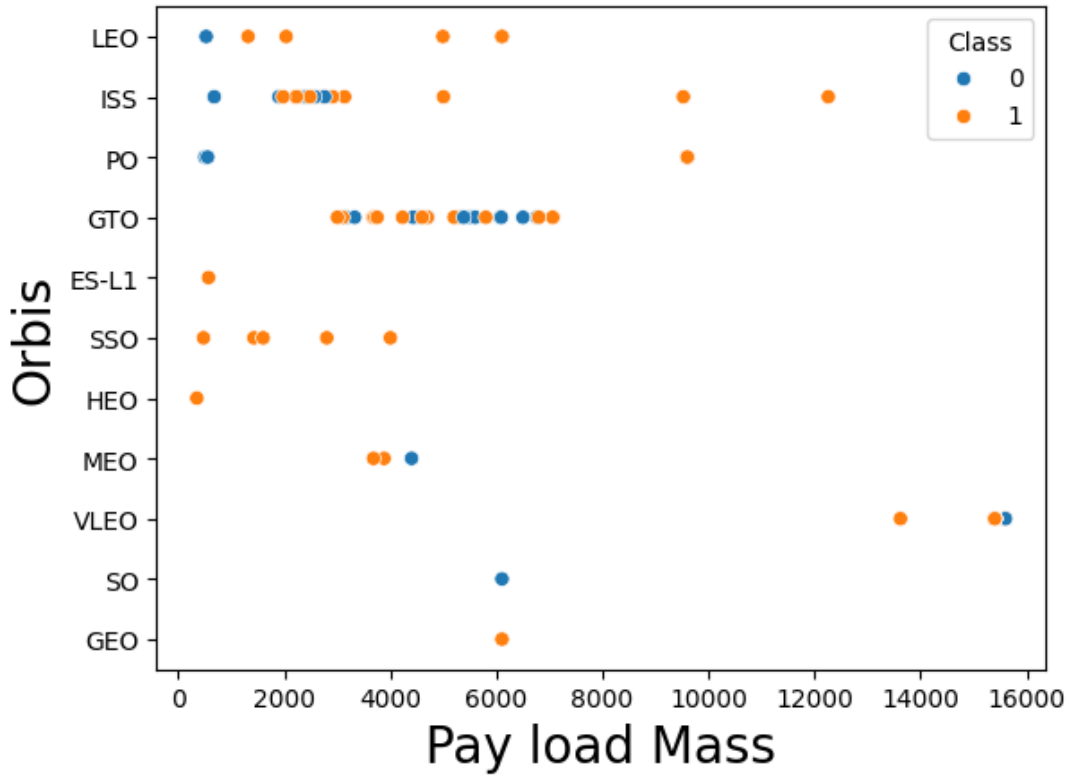
Flight Number vs. Orbit Type

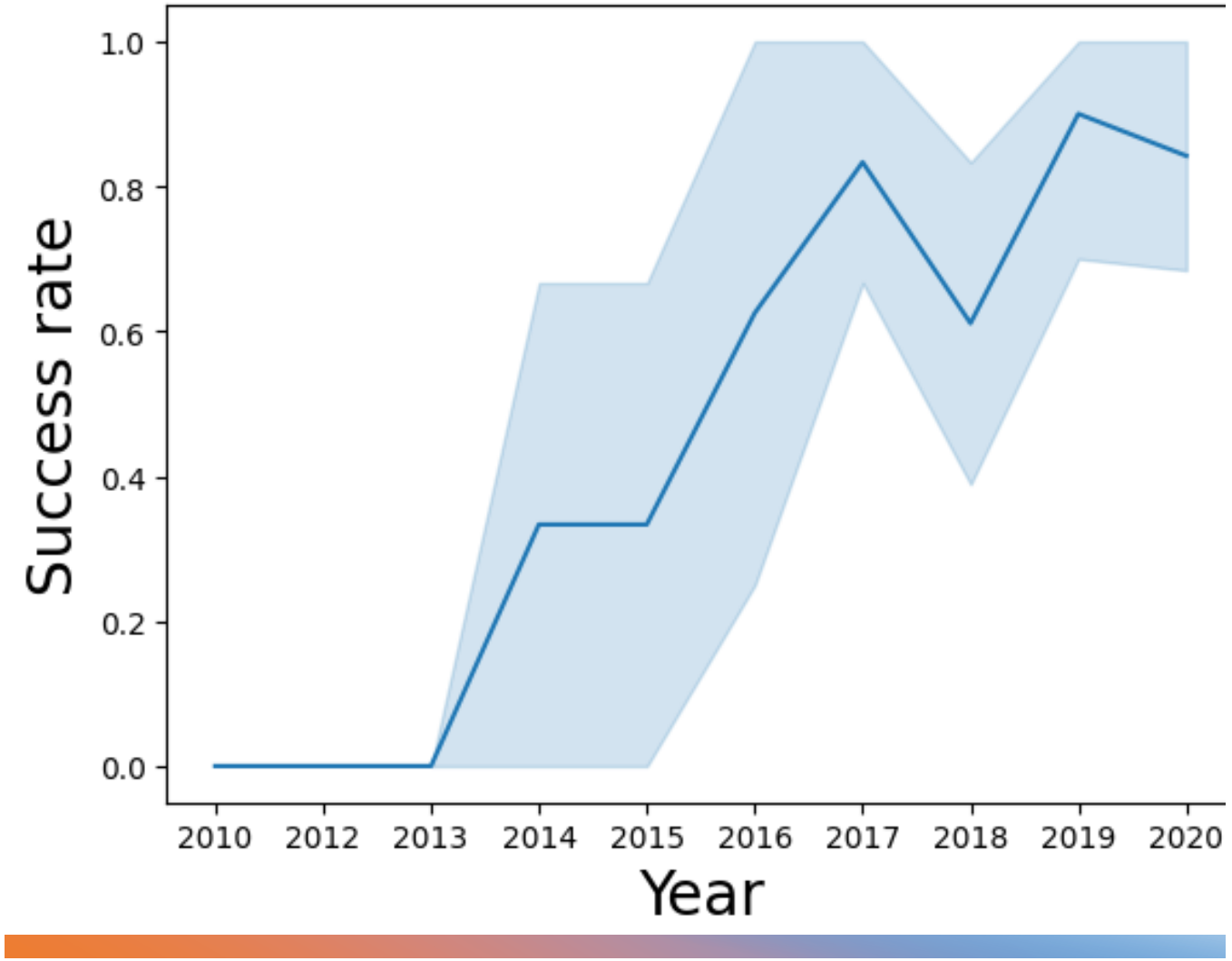
- We see that in the LEO orbit the Success appears related to the number of flights; on the other hand, there seems to be no relationship between flight number when in GTO orbit.



Payload vs. Orbit Type

- With heavy payloads the successful landing or positive landing rate are more for Polar,LEO and ISS.
- However for GTO we cannot distinguish this well as both positive landing rate and negative landing(unsuccessful mission) are both there here.





Launch Success Yearly Trend

- We can observe that the success rate since 2013 kept increasing till 2017 (stable in 2014) and after 2015 it started increasing.

All Launch Site Names

```
    rows = cur.fetchall()

[10]  ✓ 0.0s                                                    Python
```

```
    %sql  Select distinct "Launch_Site"  from SPACEXTBL

[11]  ✓ 0.0s                                                    Python
```

```
 * sqlite:///my_data1.db
Done.
```

Launch_Site
CCAFS LC-40
VAFB SLC-4E
KSC LC-39A
CCAFS SLC-40

Generate + Code + Markdown

As shown is the table there is 4 unique names af launch Site : CCAFS LC-40, VAFB SLC-4E, KSC LC-39A and CCAFS SLC-40.

Launch Site Names Begin with 'CCA'

Display 5 records where launch sites begin with the string 'CCA'

```
%%sql
select *
from SPACEXTABLE
where "Launch_Site" like 'CCA%'
limit 5
```

[22] Python

```
 * sqlite:///my_data1.db
Done.
```

Date	Time (UTC)	Booster_Version	Launch_Site	Payload	PAYLOAD_MASS__KG_	Orbit	Customer	Mission_Outcome	Landing_Outcome
2010-06-04	18:45:00	F9 v1.0 B0003	CCAFS LC-40	Dragon Spacecraft Qualification Unit	0	LEO	SpaceX	Success	Failure (parachute)
2010-12-08	15:43:00	F9 v1.0 B0004	CCAFS LC-40	Dragon demo flight C1, two CubeSats, barrel of Brouere cheese	0	LEO (ISS)	NASA (COTS) NRO	Success	Failure (parachute)
2012-05-22	7:44:00	F9 v1.0 B0005	CCAFS LC-40	Dragon demo flight C2	525	LEO (ISS)	NASA (COTS)	Success	No attempt
2012-10-08	0:35:00	F9 v1.0 B0006	CCAFS LC-40	SpaceX CRS-1	500	LEO (ISS)	NASA (CRS)	Success	No attempt
2013-03-01	15:10:00	F9 v1.0 B0007	CCAFS LC-40	SpaceX CRS-2	677	LEO (ISS)	NASA (CRS)	Success	No attempt

Total Payload Mass

Display the total payload mass carried by boosters launched by NASA (CRS)

```
%%sql
select  sum("PAYLOAD_MASS__KG_") as Total_payload_nasa_crs
from SPACEXTABLE
where Customer = 'NASA (CRS)'
```

[35] Python

```
 * sqlite:///my_data1.db
Done.
```

Total_payload_nasa_crs
45596

Generate + Code + Markdown

Average Payload Mass by F9 v1.1

Display average payload mass carried by booster version F9 v1.1

```
%%sql
select Booster_Version, avg("PAYLOAD_MASS__KG_") as avg_payload_f9v11
from SPACEXTABLE
where Booster_Version = 'F9 v1.1'
group by Booster_Version
```

[36] Python

```
 * sqlite:///my_data1.db
Done.
```

Booster_Version	avg_payload_f9v11
F9 v1.1	2928.4

First Successful Ground Landing Date

List the date when the first succesful landing outcome in ground pad was acheived.

Hint:Use min function

```
%%sql
select min(Date) as Dates_first_success_groundpad,
       "Landing_Outcome"
from SPACEXTABLE
where "Landing_Outcome" = 'Success (ground pad)'
```

[37] Python

```
 * sqlite:///my_data1.db
Done.
```

Dates_first_success_groundpad	Landing_Outcome
2015-12-22	Success (ground pad)

Generate + Code + Markdown

Successful Drone Ship Landing with Payload between 4000 and 6000

List the names of the boosters which have success in drone ship and have payload mass greater than 4000 but less than 6000

```
%%sql
select Booster_Version, "Landing_Outcome", "PAYLOAD_MASS__KG_"
from SPACEXTABLE
where "Landing_Outcome" = 'Success (drone ship)'
    and  "PAYLOAD_MASS__KG_" BETWEEN  4000 and  6000
order by Date
```

[38] Python

```
 * sqlite:///my_data1.db
Done.
```

Booster_Version	Landing_Outcome	PAYLOAD_MASS__KG_
F9 FT B1022	Success (drone ship)	4696
F9 FT B1026	Success (drone ship)	4600
F9 FT B1021.2	Success (drone ship)	5300
F9 FT B1031.2	Success (drone ship)	5200

Total Number of Successful and Failure Mission Outcomes

```
%%sql
    create table test as
    Select * ,
    (case
        when "Mission_Outcome" like "%Success%" then "Success"
        else "Failure" end)as Mission_2
    from SPACEXTBL
```

[ ] Python

```
%%sql
    select Mission_2, count(Mission_2)
    from test
    group by Mission_2
```

[15] ✓ 0.0s Python

```
 * sqlite:///my_data1.db
Done.
```

Mission_2	count(Mission_2)
Failure	1
Success	100

Boosters Carried Maximum Payload

```
%%sql select  Booster_Version , PAYLOAD_MASS__KG_
    from SPACEXTABLE
    WHERE PAYLOAD_MASS__KG_ = (select max(PAYLOAD_MASS__KG_) from SPACEXTABLE)
```

[17] ✓ 0.0s Python

```
 * sqlite:///my_data1.db
Done.
```

Booster_Version	PAYLOAD_MASS__KG_
F9 B5 B1048.4	15600
F9 B5 B1049.4	15600
F9 B5 B1051.3	15600
F9 B5 B1056.4	15600
F9 B5 B1048.5	15600
F9 B5 B1051.4	15600
F9 B5 B1049.5	15600
F9 B5 B1060.2	15600
F9 B5 B1058.3	15600
F9 B5 B1051.6	15600
F9 B5 B1060.3	15600
F9 B5 B1049.7	15600

2015 Launch Records

List the records which will display the month names, failure landing_outcomes in drone ship ,booster versions, launch_site for the months in year 2015.

Note: SQLLite does not support monthnames. So you need to use substr(Date, 6,2) as month to get the months and substr(Date,0,5)='2015' for year.

```
%%sql select substr(Date, 6,2) as Month, Landing_Outcome, Booster_Version, Launch_Site
    from SPACEXTABLE
    where Landing_Outcome ='Failure (drone ship)'
    and substr(Date, 1, 4) = '2015'
```

[18] ✓ 0.0s Python

```
 * sqlite:///my_data1.db
Done.
```

Month	Landing_Outcome	Booster_Version	Launch_Site
01	Failure (drone ship)	F9 v1.1 B1012	CCAFS LC-40
04	Failure (drone ship)	F9 v1.1 B1015	CCAFS LC-40

Rank Landing Outcomes Between 2010-06-04 and 2017-03-20

Rank the count of landing outcomes (such as Failure (drone ship) or Success (ground pad)) between the date 2010-06-04 and 2017-03-20, in descending order.

```
%%sql
    select Landing_Outcome,count(Landing_Outcome) as Rank_landing
    from SPACEXTABLE
    where date between '2010-06-04' and '2017-03-20'
    group by Landing_Outcome
    order by Rank_landing desc
```

[20] ✓ 0.0s Python

```
 * sqlite:///my_data1.db
Done.
```

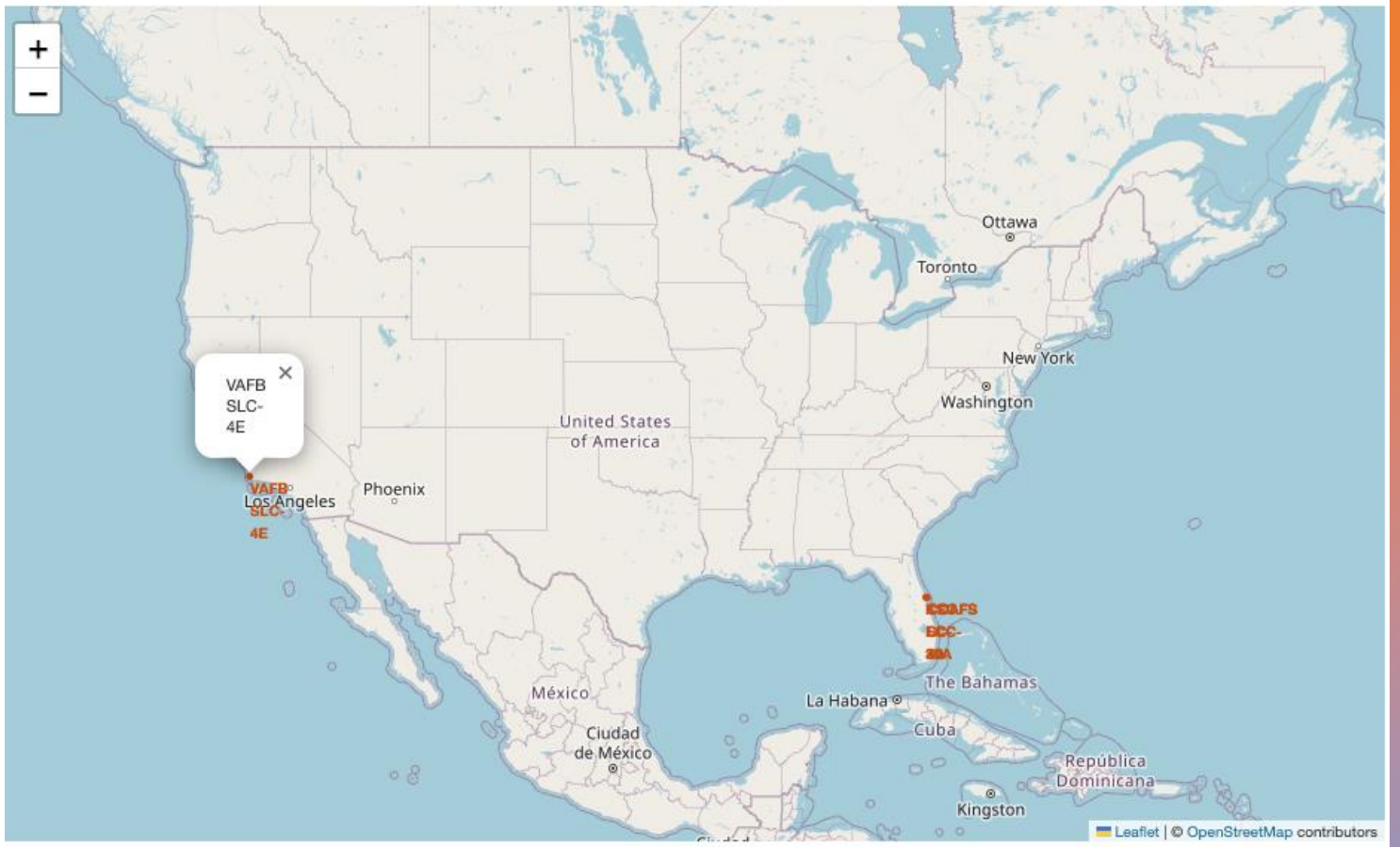
Landing_Outcome	Rank_landing
No attempt	10
Success (drone ship)	5
Failure (drone ship)	5
Success (ground pad)	3
Controlled (ocean)	3
Uncontrolled (ocean)	2
Failure (parachute)	2
Precluded (drone ship)	1

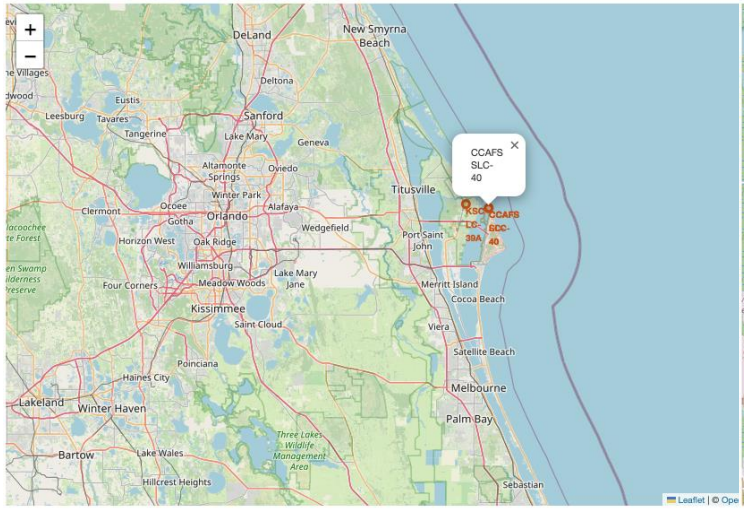
Section 3

Launch Sites Proximities Analysis

SpaceX Launch Sites – Locations and Coastal Proximities (US)

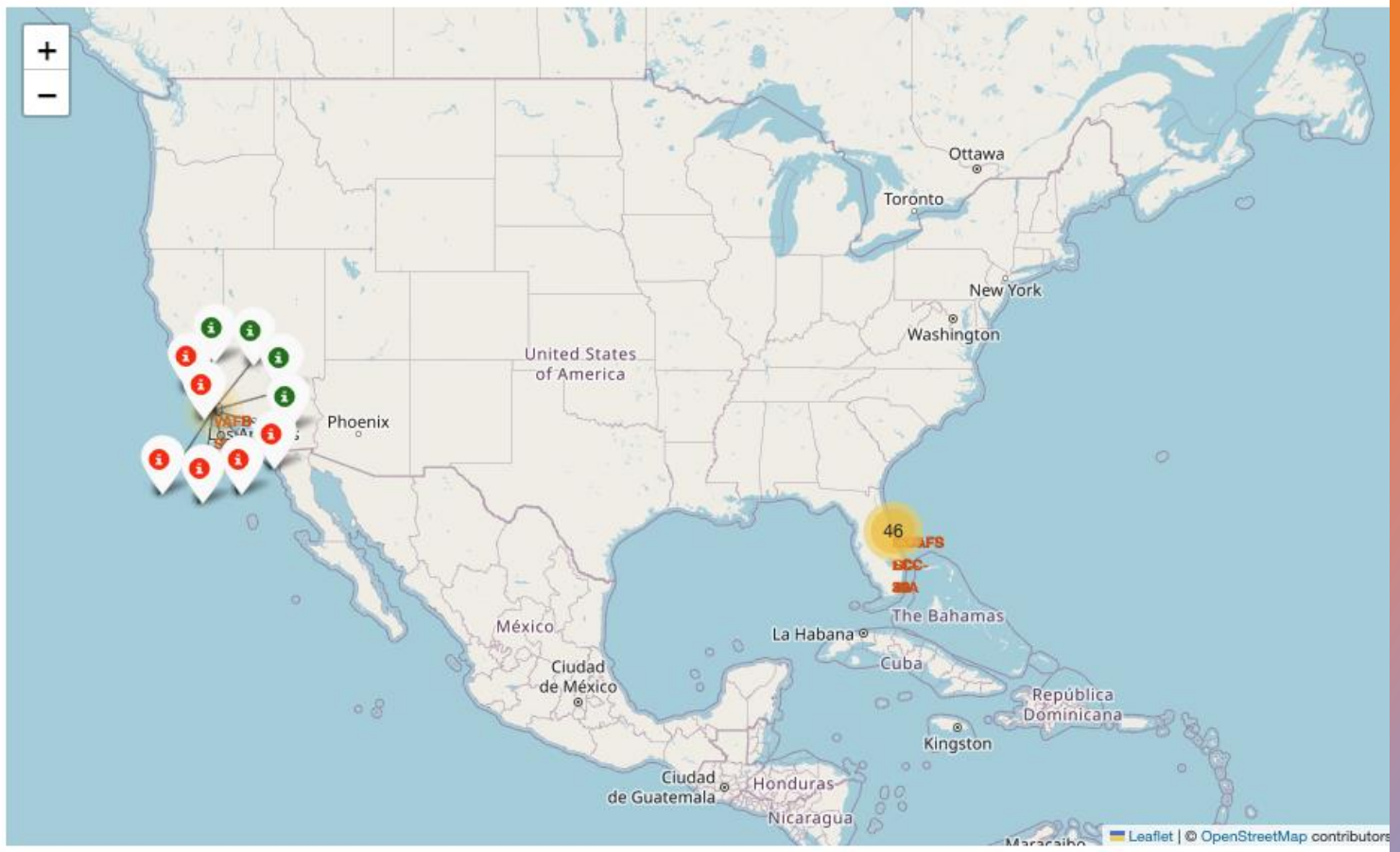
- Key findings visible from the map
  - Three launch sites are on the U.S. east coast (Cape Canaveral / KSC area) and one is on the west coast (Vandenberg AFB)
  - All four sites lie very near the coastline — consistent with coastal launch-site practice.
  - Launch sites are in proximity to the Equator line.
  - The orange circle + popup pattern highlights site locations and allows inspection when the map is interactive.

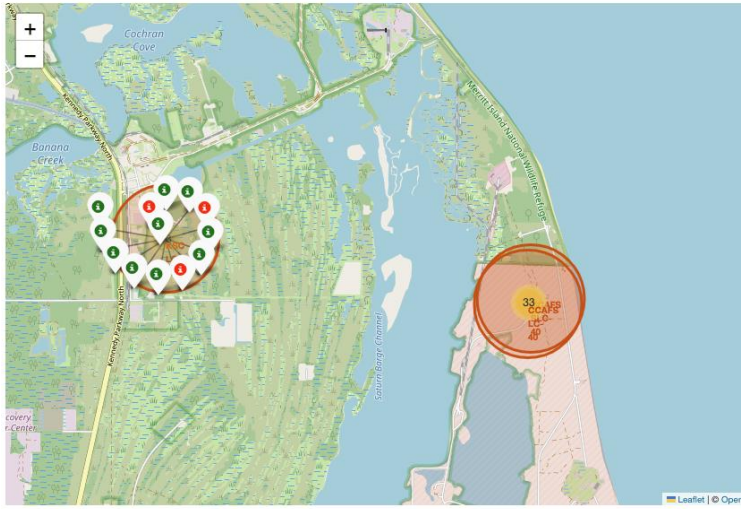


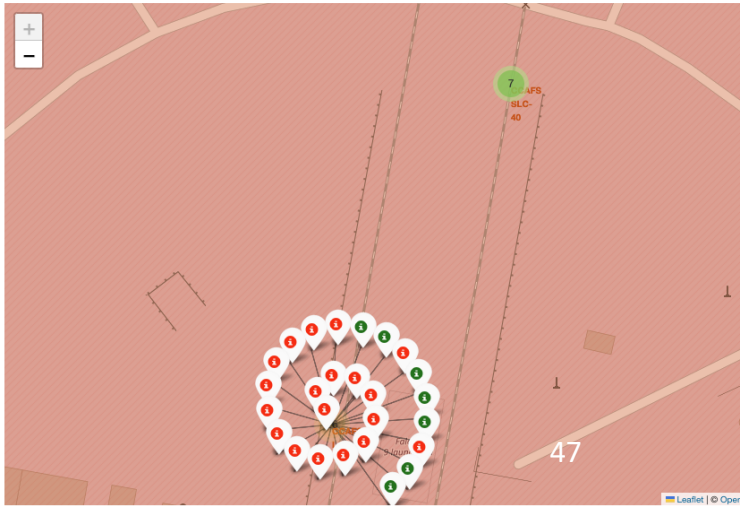


Launch sites outcomes(success/failure)

I add the launch outcomes for each site to see which sites have high success rates. Each launch record is plotted in a marker cluster with color coding: green = success, red = failure.

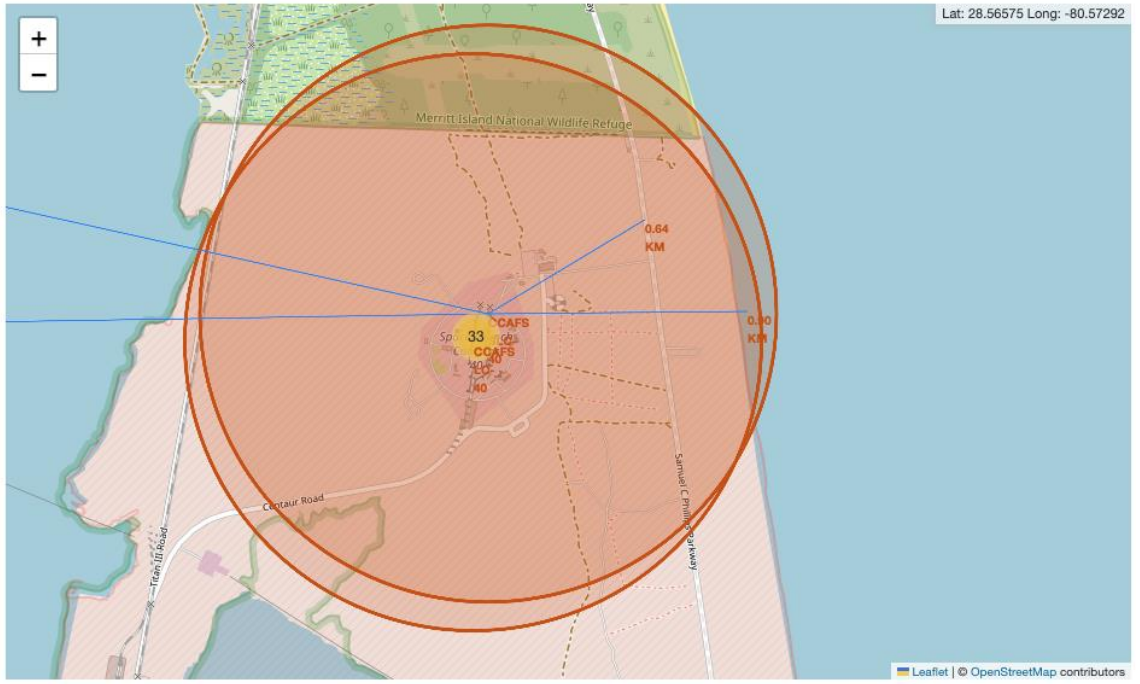


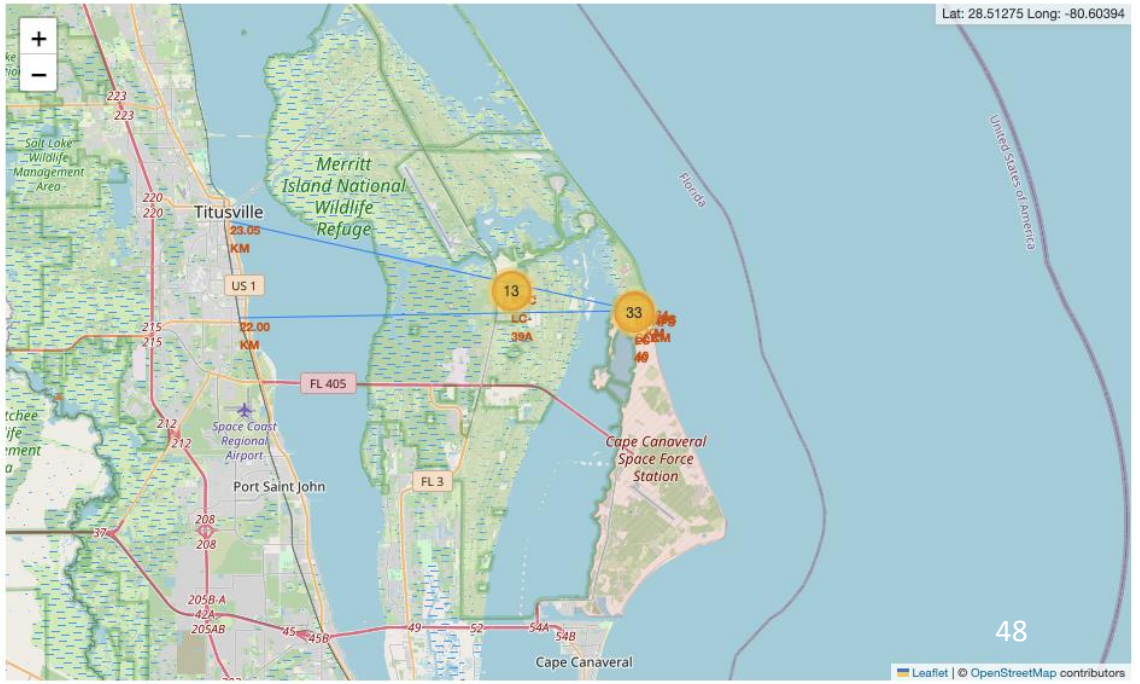




CCAFS SLC-40 Launch site distance from railway, highway and city.

- Railways: some launch sites have railways within a few kilometers, others do not. Visual inspection alone is inconclusive; compute distances to confirmed railway coordinates to be sure.
- Highways: most sites show highways nearby (within a few km) on the plotted lines.
- Coastline: all launch sites are sited very close to the coast (typically a few kilometers or less).
- Cities: Launch sites are kept out of dense urban centers — usually several kilometers (often 10+ km) from city centers for safety and range clearance





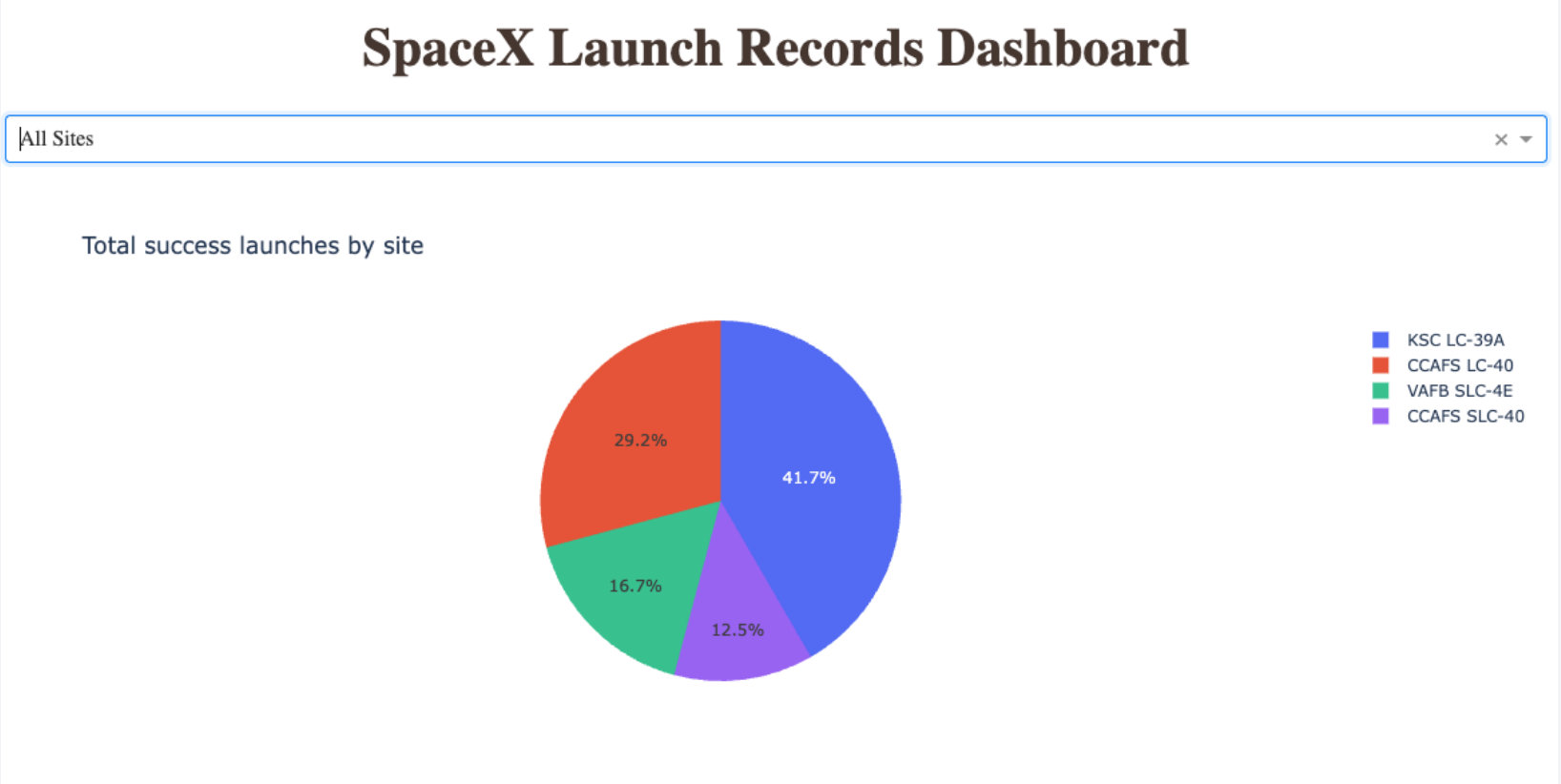
Section 4

Build a Dashboard with Plotly Dash

Pie chart of launch success rate for all sites

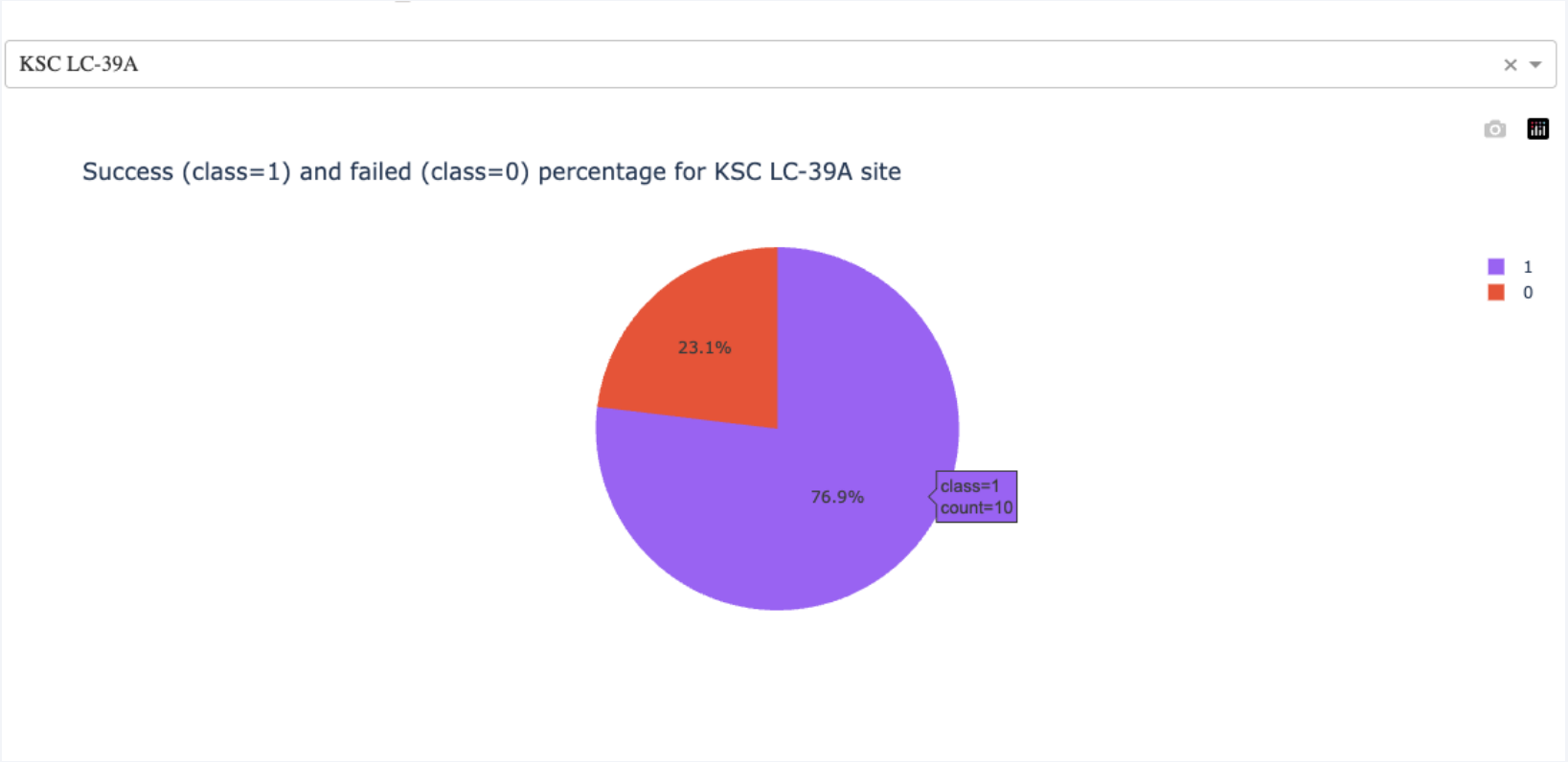
Findings :

The launch site with the highest success outcome is KSC LC-39A, with over 41% of success rate.
Followed by the CCAFS LC-40, with 29.2% success rate.

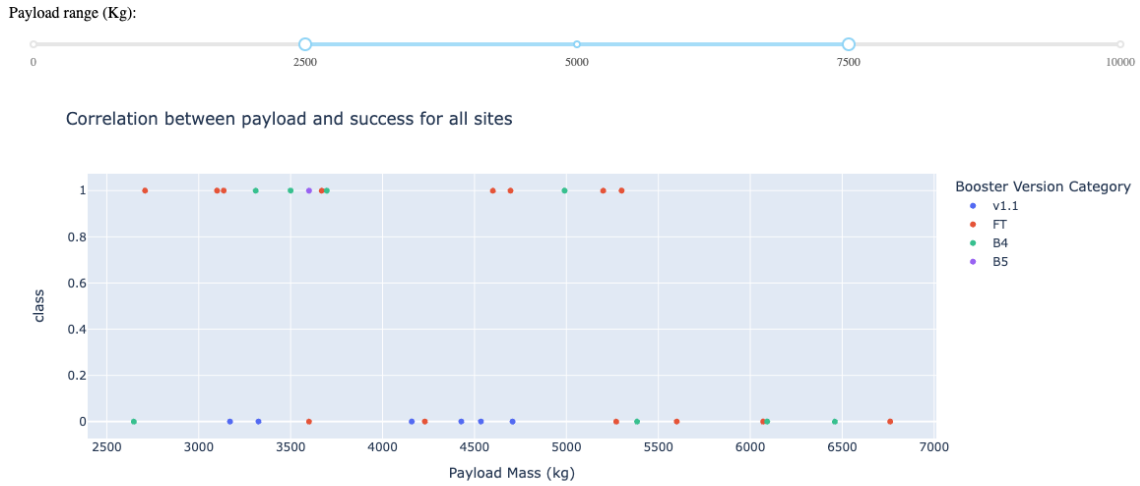


Pie chart of the highest success launch site

Findings:
The highest success launch site account 10 success outcome over 13, that is 76.9% of success occurred on launch at the KSC LC – 39A launch site.



Payload vs. Launch Outcome scatter plot for all sites



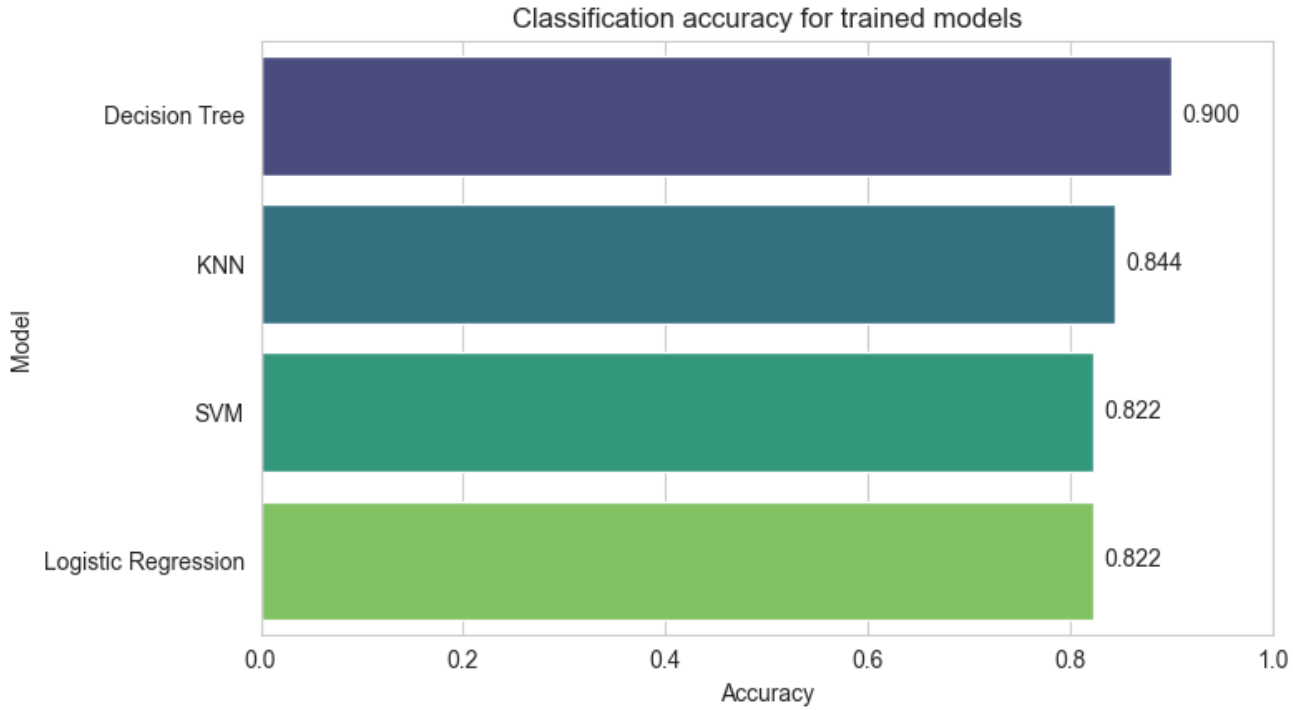
Findings:
The largest success rate payload range is 2500-7500, with FT booster version.

Section 5

Predictive Analysis (Classification)

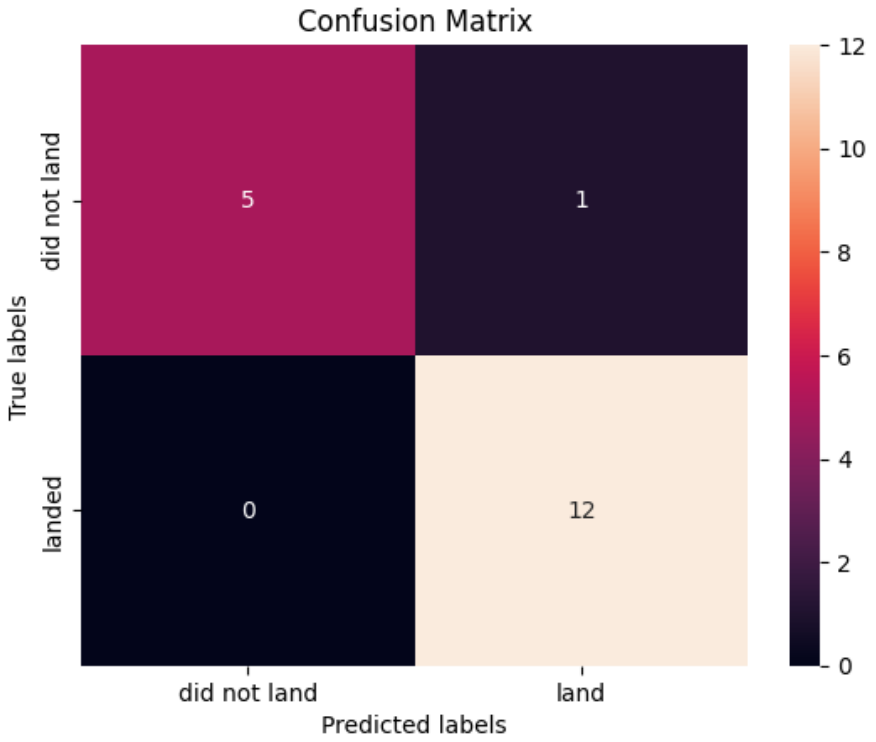
Classification Accuracy

- Best performing model is Decision Tree with 0.9 accuracy on training data set



Confusion Matrix

- The model is very accurate on the test set (≈94%).
- It never missed a true landing (recall = 100%): no false negatives — good if missing landed missions is costly.
- There is one false positive (model predicted a landing where none occurred), so the model slightly over-predicts "land".
- Specificity is lower (~83%): the model is less reliable at correctly identifying non-landings.



Conclusions

- The objective of this project is to predict whether a Falcon 9 first stage will land using features. The project demonstrates that launch site, payload mass, and booster version are significant predictors of Falcon 9 landing success.
- Best performing model is Decision Tree with 0.9 accuracy on training data set
- Interactive dashboards and geospatial maps provide valuable insights for mission planning. Predictive models can support decision-making and risk assessment for future launches.
- The launch site with the highest success outcome is KSC LC-39A, with over 41% of success rate. Followed by the CCAFS LC-40, with 29.2% success rate.

Appendix

- Include any relevant assets like Python code snippets, SQL queries, charts, Notebook outputs, or data sets that you may have created during this project

  - Data Sources: SpaceX API, Wikipedia, provided CSVs.
  - Tools & Libraries: Python, Pandas, NumPy, BeautifulSoup, Requests, SQLite, Folium, Plotly, Dash, Scikit-learn.
  - Project Artifacts by Week:
  - Week 1: Data collection and wrangling notebooks.
  - Week 2: SQL analysis and EDA.
  - Week 3: Geospatial mapping and dashboard app.
  - Week 4: Machine learning modeling and evaluation.
  - Code samples available in respective week folders.

Thank you!